Lab 6
2. Game Development
Concepts in Java

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Tuesday, February 24th, 2026 | | ER 3120

University of Windsor

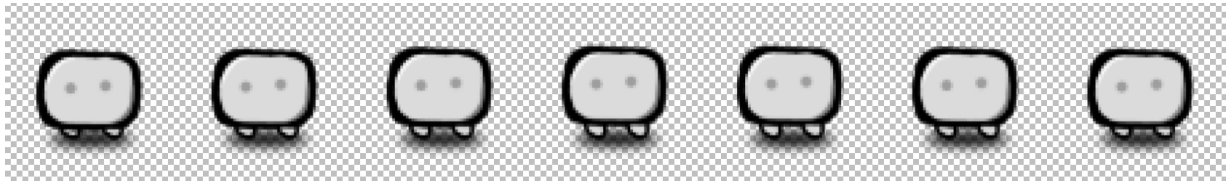
Part I
Sprite Animation

University of Windsor

Sprite Animation

University of Windsor

Sprite Animation



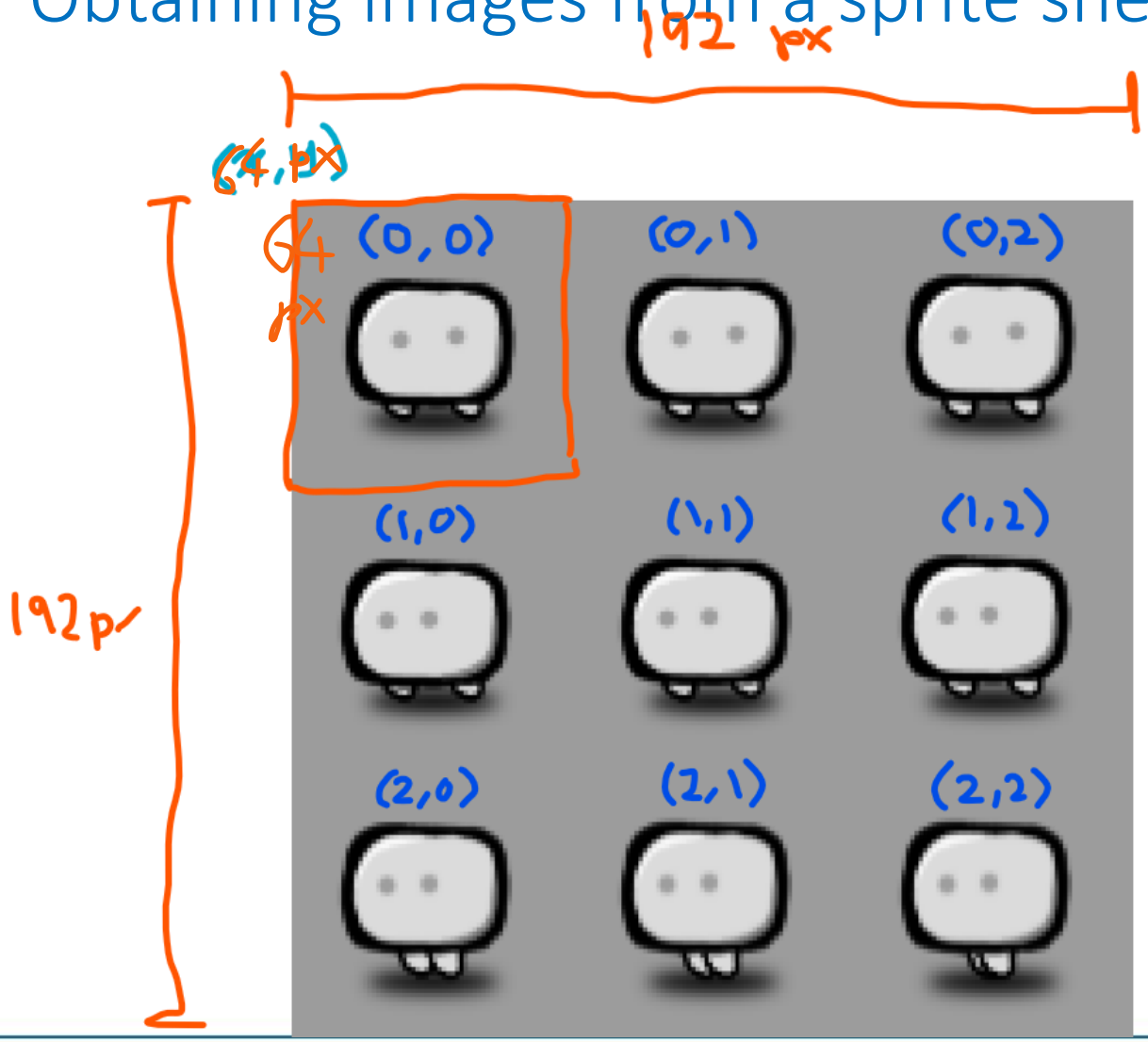
- In 2D games, animations like a character running is typically done through showing a group of images in succession and then looping them.
- A sprite sheet (the one on the right) at its core is just an array of images that are obtained from one big source image.

Creating an `ImageLoader` class

- We want to make it easy to load images from our computer's memory.
- So, we create a separate utility class to load images.

```java
import java.awt.image.*;
import javax.imageio.*;
import java.io.*;

public class ImageLoader {

    public static BufferedImage loadImage(String filepath) {
        BufferedImage image = null;
        try {
            image = ImageIO.read(new File(filepath));
        } catch (IOException e) {
            System.out.println(filepath + " was not found!");
        }

        return image;
    }

}
```

Obtaining images from a sprite sheet



$x = (\text{column}) \times (\text{cell width})$

$y = (\text{row}) \times (\text{cell height})$

University of Windsor

Creating a `SpriteSheet` class

- We create a `SpriteSheet` class which will represent a sprite sheet in our game.
- It takes in the source image and then extracts all the sub images from it based on the rows and columns and the cell dimensions.

Creating a `SpriteSheet` class

```java
import java.awt.image.BufferedImage;

public class SpriteSheet {

    public BufferedImage[] images;
    public BufferedImage source;

    public SpriteSheet(String filepath, int rows, int columns, int cellWidth, int cellHeight) {
        source = ImageLoader.loadImage(filepath);
        images = new BufferedImage[rows * columns];

        int index = 0;
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < columns; c++) {
                int x = c * cellWidth;
                int y = r * cellHeight;
                images[index] = source.getSubimage(x, y, cellWidth, cellHeight);
                index++;
            }
        }
    }
}
```

Animators

University of Windsor

Creating an `Animator`

- We create an `Animator` class which takes in the array of images/frames that will be used to run the animation. We also take in a `startIndex` which will determine from which frame in the array we want to start our animation. The `durationPerFrame` is how much time we want to spend on each frame before we move onto the next. Current index keeps track of the index of the current frame.
- Based on our game loop, the time units here are approximately 1/60th of a second. The time variable keeps track of the time spent on ever frame and when it reaches the `durationPerFrame`, we switch to the next frame.

Creating an `Animator`

- We create an `Animator` images/frames that will take in a `startIndex` w in the array we want to s `durationPerFrame` is each frame before we m track of the index of the
- Based on our game loop, 1/60th of a second. The spent on ever frame and `durationPerFrame`, w

```java
import java.awt.image.BufferedImage;

public class Animator {

    public BufferedImage[] frames;
    public BufferedImage currentFrame;

    private int time = 0;
    public int currentIndex;
    public int durationPerFrame;

    public Animator(BufferedImage[] frames, int startIndex, int durationPerFrame) {
        this.frames = frames;
        this.currentIndex = startIndex;
        this.durationPerFrame = durationPerFrame;
        currentFrame = frames[currentIndex];
    }

    public void tick() {
        time++;
        if(time >= durationPerFrame) {
            time = 0;
            currentIndex++;
            if(currentIndex > frames.length - 1)
                currentIndex = 0;
        }

        currentFrame = frames[currentIndex];
    }
}
```

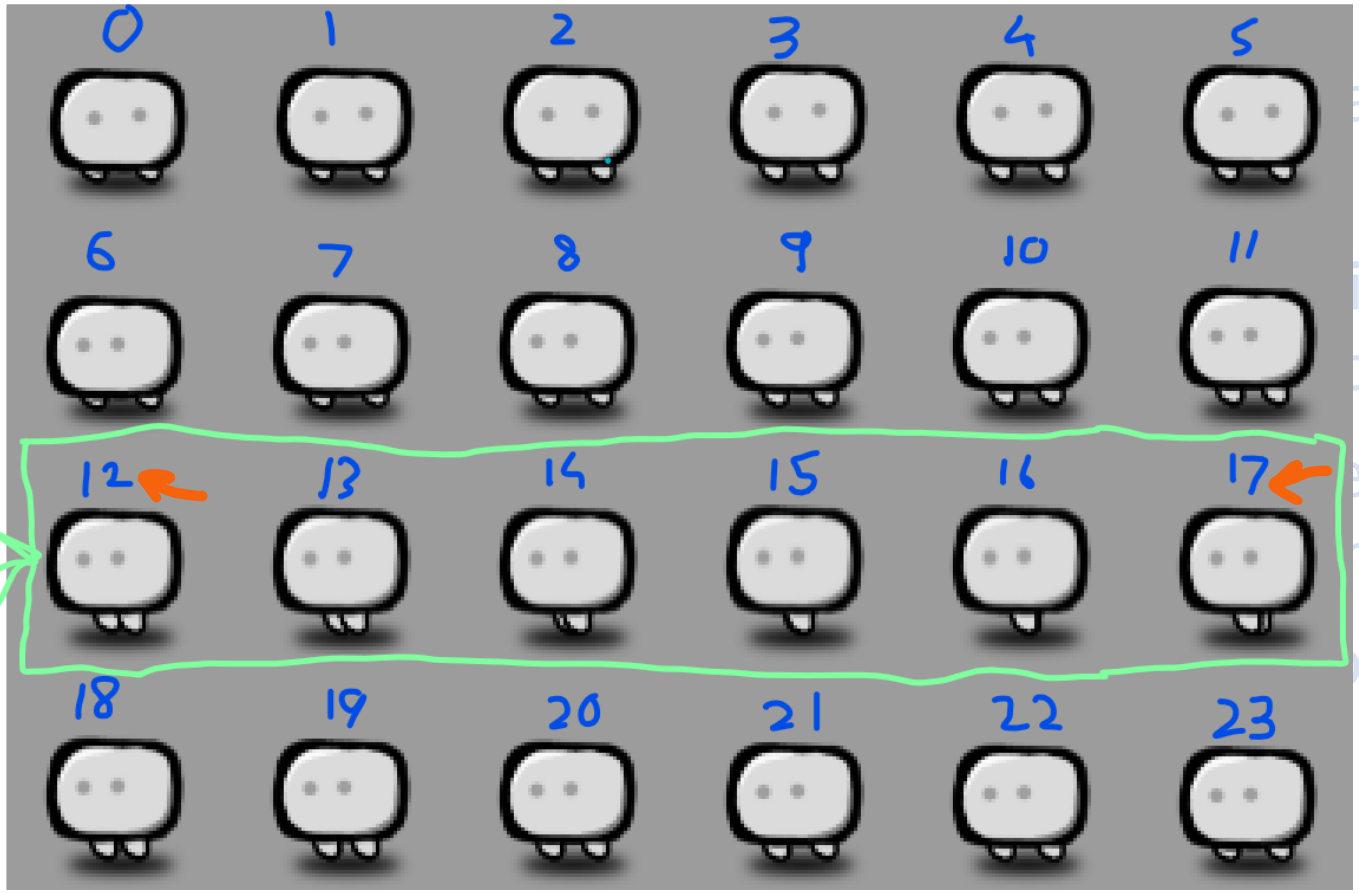
Sprite Flexibility

University of Windsor

Making our Sprite sheet more flexible

- Our sprite sheet contains the image array that contains all sub images in our sprite sheet.
- Let’s say one of the rows contains idle/standing animation and some other row contains the running animation.
- When we create an animator for walking, we can’t simply pass all our images from the sprite sheet to the animator.
- We only need to pass in the images which contain our run animation.

Making our Sprite sheet more flexible



- at contains all sub
- ing animation and tion.
- e can't simply pass nimator.
- ntain our run

University of Windsor

Making our Sprite sheet more flexible

- We can add a method to our `SpriteSheet` class which will extract our desired images using a start and an end index.
- It will return our desired sub array.

```
public BufferedImage[] getImagesFrom(int startIndex, int endIndex) {
    return Arrays.copyOfRange(images, startIndex, endIndex + 1);
}
```

Game Object

University of Windsor

Game Object

- Everything in a game world falls under the category of a `Game Object`. So, when programming, we want this hierarchy.
- We can create a Class called `GameObject` and every other object like `Player`, `Enemy`, `Bullets`, `Platforms`, `Blocks`, etc. will derive from our `GameObject` class.
- Every `GameObject` needs to have their respective `tick()` and `render()` methods which will control how they are updated and drawn. In addition to that, it also needs to have an x and y position.
- In more sophisticated games, a more advanced system is used, called `Entity Component System` (ECS), but for our purposes we can ignore this.

`GameObject` class

- On its own, this class does not serve any purpose. We need to derive classes from it to make it useful.
- The `render()` method takes in a graphics context which will allow us to draw graphics.

```java
import java.awt.Graphics2D;

public class GameObject {

    public int x;
    public int y;

    public GameObject(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public void tick() {}
    public void render(Graphics2D g2d) {}
}
```


Creating a `Player GameObject`

- We created a Player class which is a child of `GameObject` which makes it a `GameObject` at root.
- It overrides the `tick()` and `render()`.
- The `Player` is now represented by a green square placed at `x` and `y` location.

```java
import java.awt.Color;
import java.awt.Graphics2D;

public class Player extends GameObject {

    public Player(int x, int y) {
        super(x, y);
    }

    @Override
    public void tick() {

    }

    @Override
    public void render(Graphics2D g2d) {
        g2d.setColor(Color.green);
        g2d.fillRect(x, y, 64, 64);
    }
}
```


Part II
Game Manager

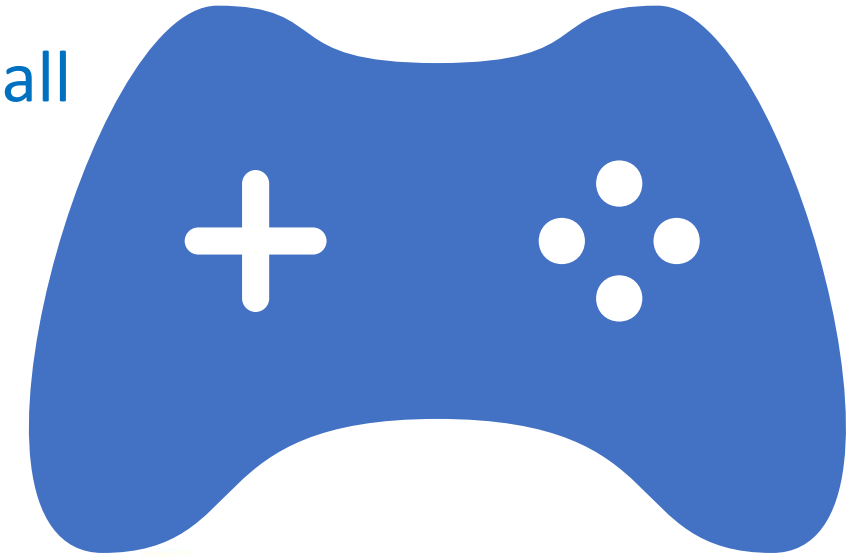
University of Windsor

Game Manager

University of Windsor

Creating a Game Manager

- A Game Manager's purpose is to store all the game objects currently in the world.
- If we want to add or remove something from the world, we must use the Game Manager.
- It is also responsible for ticking and rendering all the game objects in the world.



Game Manager

- The Game Manager keeps a list of all the game objects currently in the game.
- Then, it goes through every game object in the list and ticks it and renders it.
- There are also methods to add or remove a game object from the list.
- So, anything that's removed from the list won't be rendered or ticked.

```java
import java.awt.Graphics2D;
import java.util.ArrayList;

public class GameManager {

    public ArrayList<GameObject> gameObjects = new ArrayList<GameObject>();

    public void tick() {
        gameObjects.forEach(gameObject -> gameObject.tick());
    }

    public void render(Graphics2D g2d) {
        gameObjects.forEach(gameObject -> gameObject.render(g2d));
    }

    public void addGameObject(GameObject go) {
        gameObjects.add(go);
    }

    public void removeGameObject(GameObject go) {
        gameObjects.remove(go);
    }
}
```

Using the Game Manager in the Game Canvas

```
private GameManager gameManager = new GameManager();
```

```
private void tick() {
    gameManager.tick();
}

public void render() {
    Graphics2D g2d = (Graphics2D) bs.getDrawGraphics();
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    g2d.setColor(Color.black);
    g2d.fillRect(0, 0, this.getWidth(), this.getHeight());

    gameManager.render(g2d);

    g2d.dispose();
    bs.show();
}
```

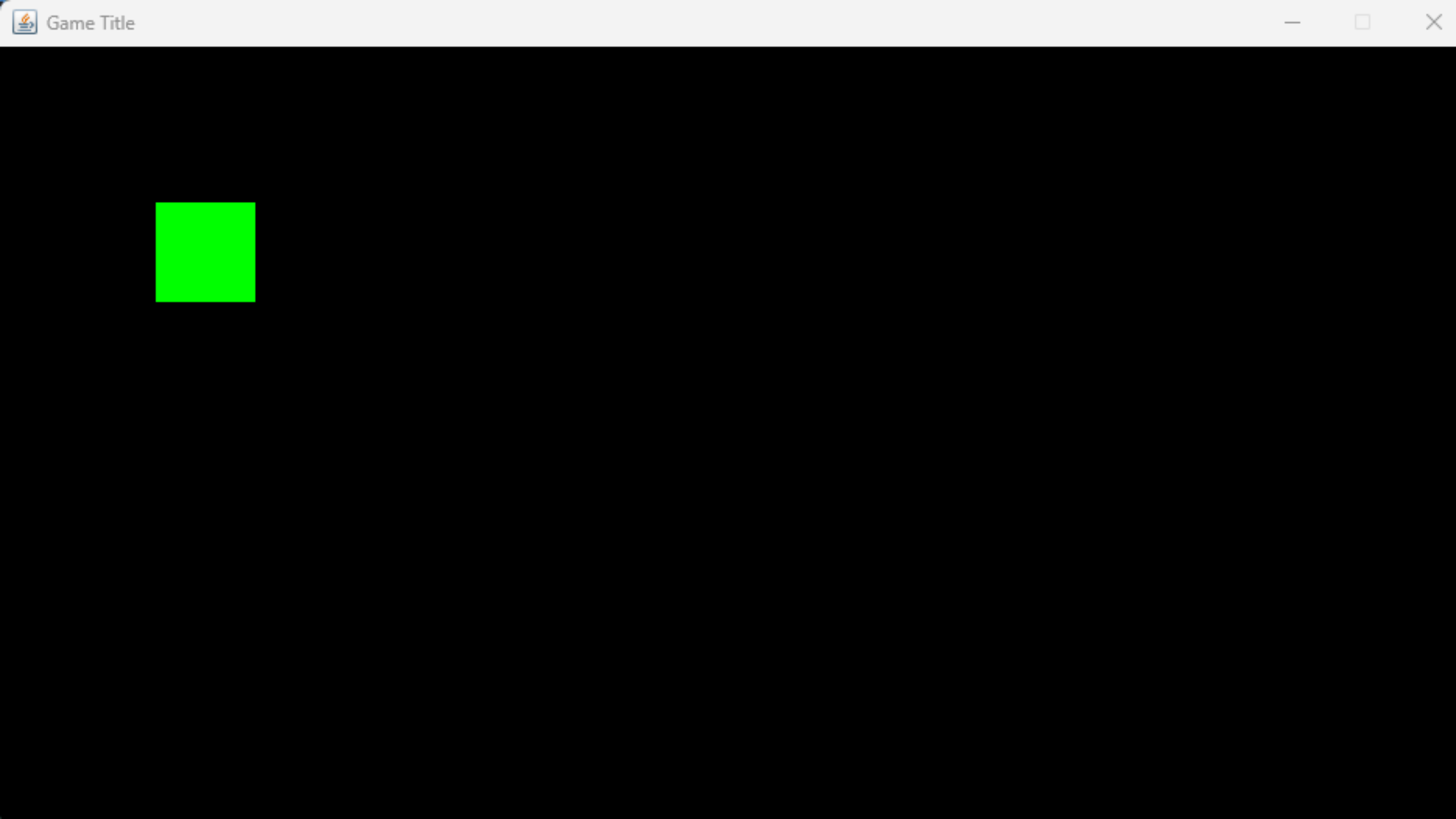
Player

University of Windsor

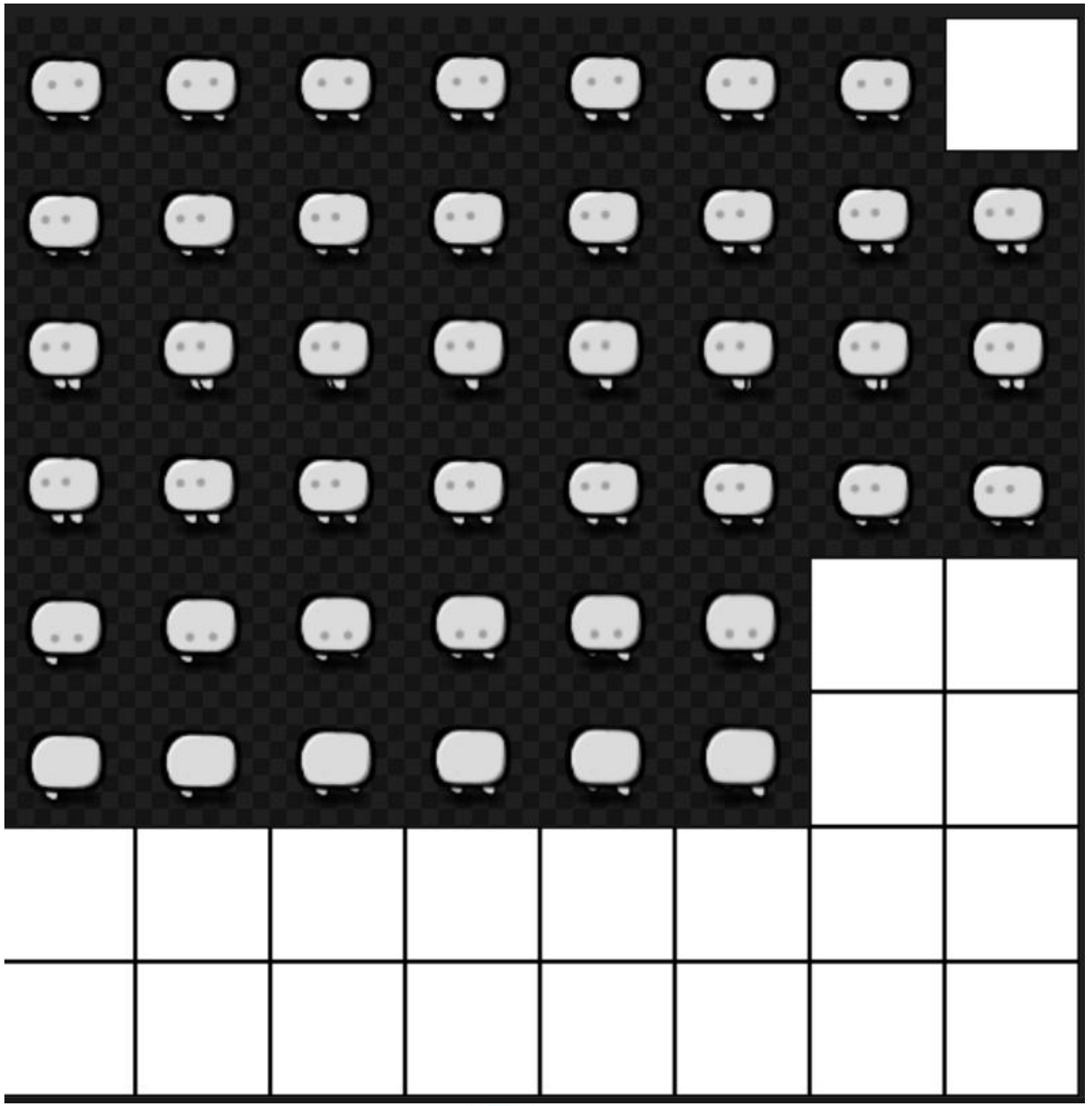
Adding a Player to our world

- We simply instantiate a `Player` and add it to our game world using the game manager.
- If we run the program, we should see our player on the screen.

```
private GameManager gameManager = new GameManager();
private Player player = new Player(100, 100);

public GameCanvas() {
    gameManager.addGameObject(player);
}
```


Game Title

PLAYER Sheet



Resources

Creating a Resources Class

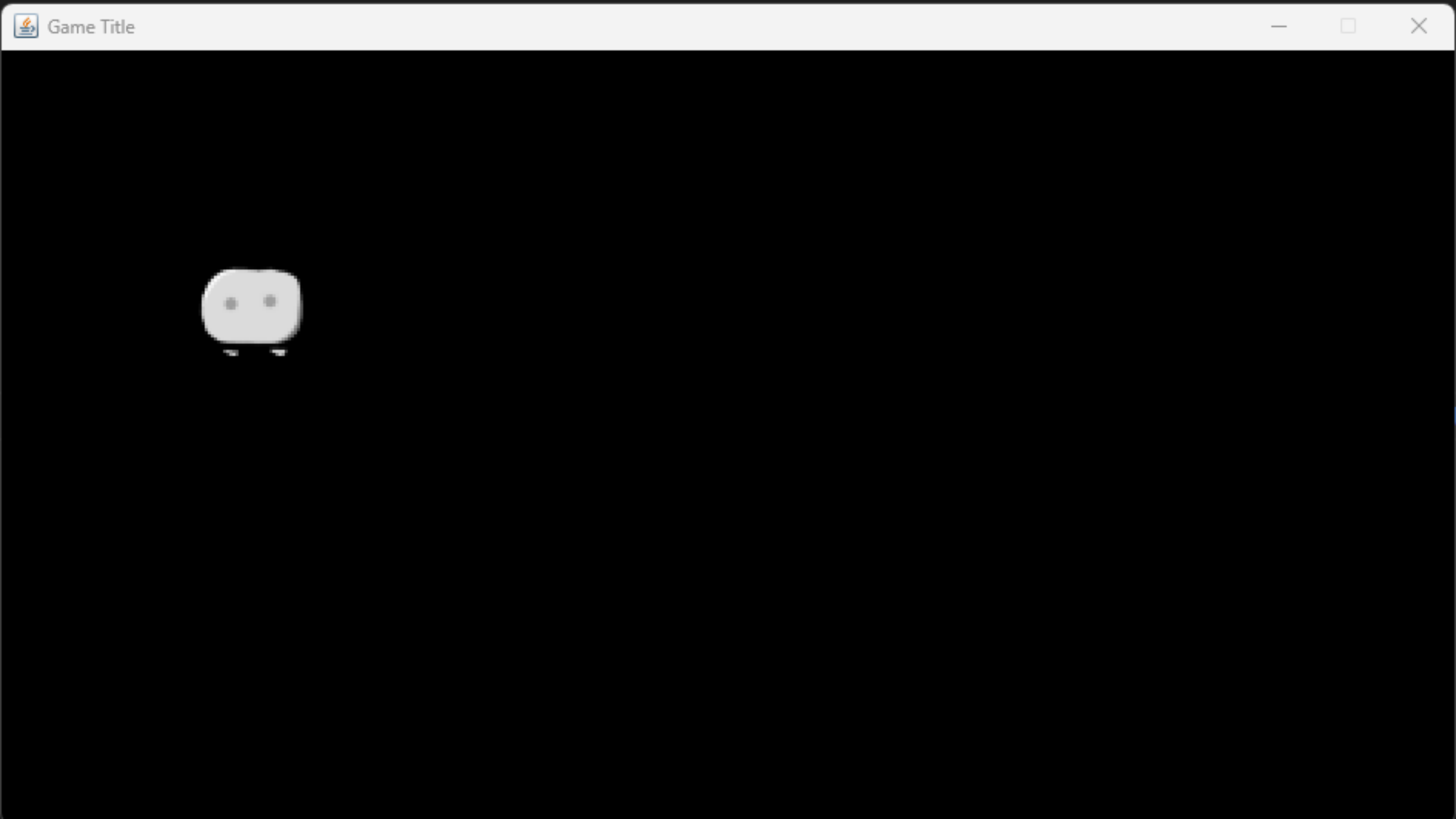
- Imagine we have 3 player objects in our game, that are using the same sprite sheet.
- It is not efficient to load the sprite sheet in the Player constructor.
- Instead, we can simply load the image once when the game starts and then we can reuse it however many times we want.
- For this reason, we create a Resources class which will contain all our images and perhaps even other loaded files.
- We make the sheet static which means there is only one instance of it across the program.

```
public class Resources {

    public static SpriteSheet playerSheet = new SpriteSheet("res/PlayerSheet.png", 8, 8, 64, 64);

}
```

Drawing image for the Player

- In the Player class we can use the sprite sheet and get the first sub image in it to represent our player for now.

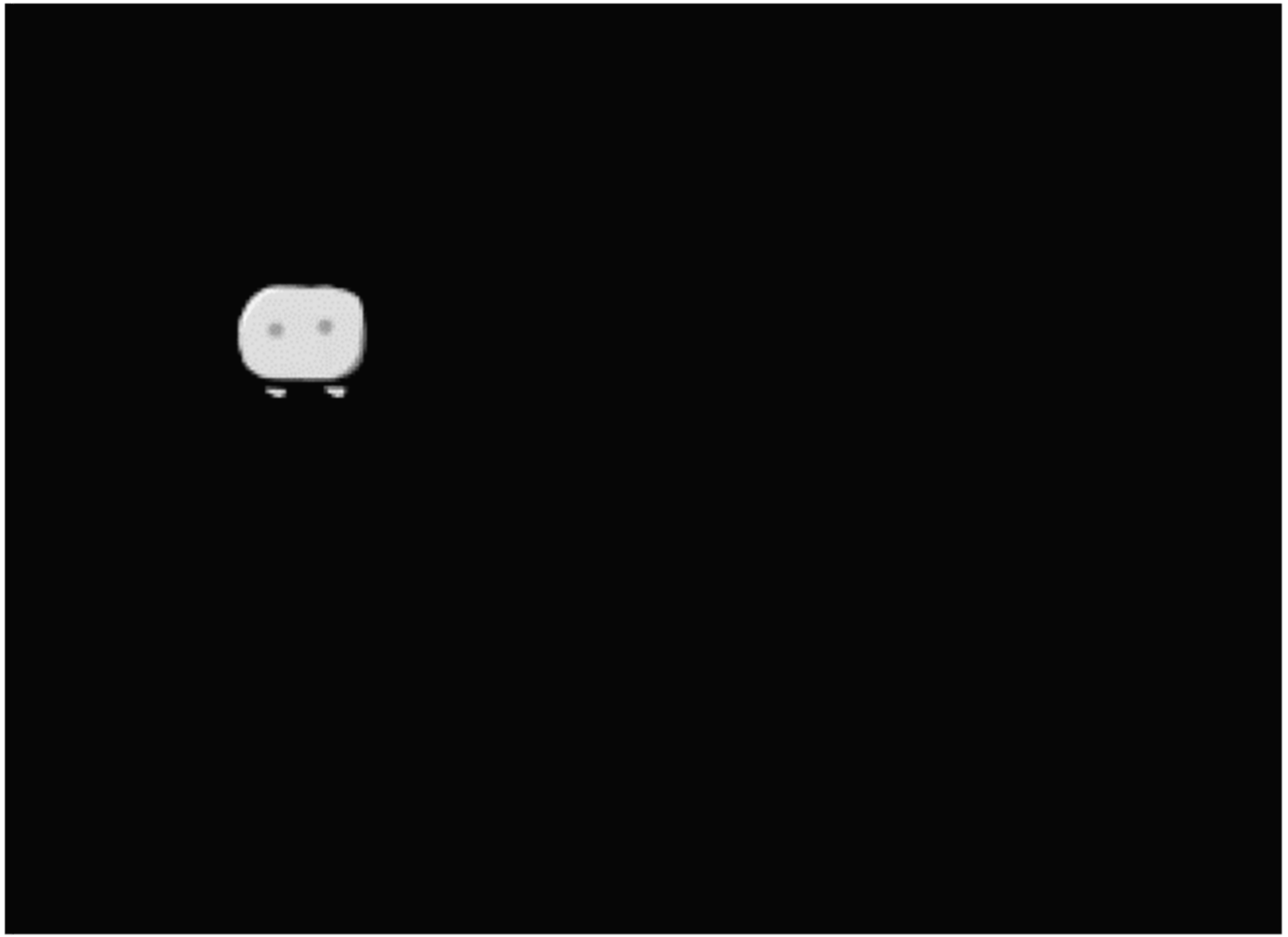
```
@Override
public void render(Graphics2D g2d) {
    g2d.drawImage(Resources.playerSheet.images[0], x, y, 128, 128, null);
}
```


Game Title

Adding idle animation to the Player

- We can use our Animator class to create an idle animation for the Player.

```
public class Player extends GameObject {

    private Animator idleAnimator;

    public Player(int x, int y) {
        super(x, y);

        BufferedImage[] imagesForIdleAnimation = Resources.playerSheet.getImagesFrom(0, 6);
        idleAnimator = new Animator(imagesForIdleAnimation, 0, 5);
    }

    @Override
    public void tick() {
        idleAnimator.tick();
    }

    @Override
    public void render(Graphics2D g2d) {
        g2d.drawImage(idleAnimator.currentFrame, x, y, 128, 128, null);
    }
}
```



Player movement

- We create a Keyboard class with 4 public variables which will representing the WASD keys.

```java
import java.awt.event.*;

public class Keyboard extends KeyAdapter {

    public boolean left = false;
    public boolean right = false;
    public boolean up = false;
    public boolean down = false;

    @Override
    public void keyPressed(KeyEvent e) {
        int keyCode = e.getKeyCode();
        if (keyCode == KeyEvent.VK_A) left = true;
        else if (keyCode == KeyEvent.VK_D) right = true;
        else if (keyCode == KeyEvent.VK_W) up = true;
        else if (keyCode == KeyEvent.VK_S) down = true;
    }

    @Override
    public void keyReleased(KeyEvent e) {
        int keyCode = e.getKeyCode();
        if (keyCode == KeyEvent.VK_A) left = false;
        else if (keyCode == KeyEvent.VK_D) right = false;
        else if (keyCode == KeyEvent.VK_W) up = false;
        else if (keyCode == KeyEvent.VK_S) down = false;
    }
}
```

Keyboard

University of Windsor

Adding Keyboard to the Game Canvas

```
private GameManager gameManager = new GameManager();
private Player player = new Player(100, 100);

public static Keyboard keyboard = new Keyboard();

public GameCanvas() {
    this.addKeyListener(keyboard);
    gameManager.addGameObject(player);
}

public void start() {
    this.requestFocus();
    this.createBufferStrategy(2);
    bs = this.getBufferStrategy();
    thread = new Thread(this, "Game Thread");
    thread.start();
}
```

Player movement continued

- Since `Keyboard` is a static member of `GameCanvas`, we can use it directly in our `Player` class. In the tick method we can move the player based on keyboard input.
- We draw the player image to 128x128 size to make it bigger. Its original size is 64x64.

```
public class Player extends GameObject {

    private Animator idleAnimator;

    public Player(int x, int y) {
        super(x, y);

        BufferedImage[] imagesForIdleAnimation = Resources.playerSheet.getImagesFrom(0, 6);
        idleAnimator = new Animator(imagesForIdleAnimation, 0, 5);
    }

    @Override
    public void tick() {
        if(GameCanvas.keyboard.left) x -= 1;
        if(GameCanvas.keyboard.right) x += 1;
        if(GameCanvas.keyboard.up) y -= 1;
        if(GameCanvas.keyboard.down) y += 1;

        idleAnimator.tick();
    }

    @Override
    public void render(Graphics2D g2d) {
        g2d.drawImage(idleAnimator.currentFrame, x, y, 128, 128, null);
    }
}
```

Camera

University of Windsor

Camera

- A Camera is also a game object. A Camera however doesn't need to render anything since it doesn't represent something that we can see. It does however need to be updated every frame.
- The Camera class has a target property which represents the game object that the Camera should follow. Hence every tick we are setting the Camera's position to be the target's position.
- This will set the Camera's position (top-left) to the player's position (top-left). But we want the player to be centered in the Camera's view. We will fix this later.

Camera

- A Camera is
need to ren
that we can
frame.
- The Camera
game objec
we are setti
- This will set
position (to
the Camera

```java
public class Camera extends GameObject {

    private GameObject target;

    public Camera(GameObject target) {
        super(target.x, target.y);
        this.target = target;
    }

    @Override
    public void tick() {
        this.x = target.x;
        this.y = target.y;
    }
}
```

doesn't
omething
d every
ents the
very tick
t's position.
ayer's
tered in

Camera

```
public static Camera camera;

public GameCanvas() {
    camera = new Camera(player);
    gameManager.addGameObject(player);
    gameManager.addGameObject(camera);
}
```

Camera

- In any 2D/3D game, whenever a `Camera` moves in some direction, everything in the game world moves in the opposite direction. This simulates a "`Camera`".
- So, in Game Manager, right before we draw all the game objects we translate everything in the opposite direction of the Camera.

```
public void render(Graphics2D g2d) {
    g2d.translate(-GameCanvas.camera.x, -GameCanvas.camera.y);
    gameObjects.forEach(gameObject -> gameObject.render(g2d));
}
```


Better Camera movement

- This code will ensure that the Camera has the target centered in its view.
- We offset the Camera's position by half of window's width and height.
- We also include half of Player's width and height (128/2 = 64) to ensure that its perfectly centered.

```
@Override
public void tick() {
    this.x = target.x - (960 / 2) + 64;
    this.y = target.y - (540 / 2) + 64;
}
```

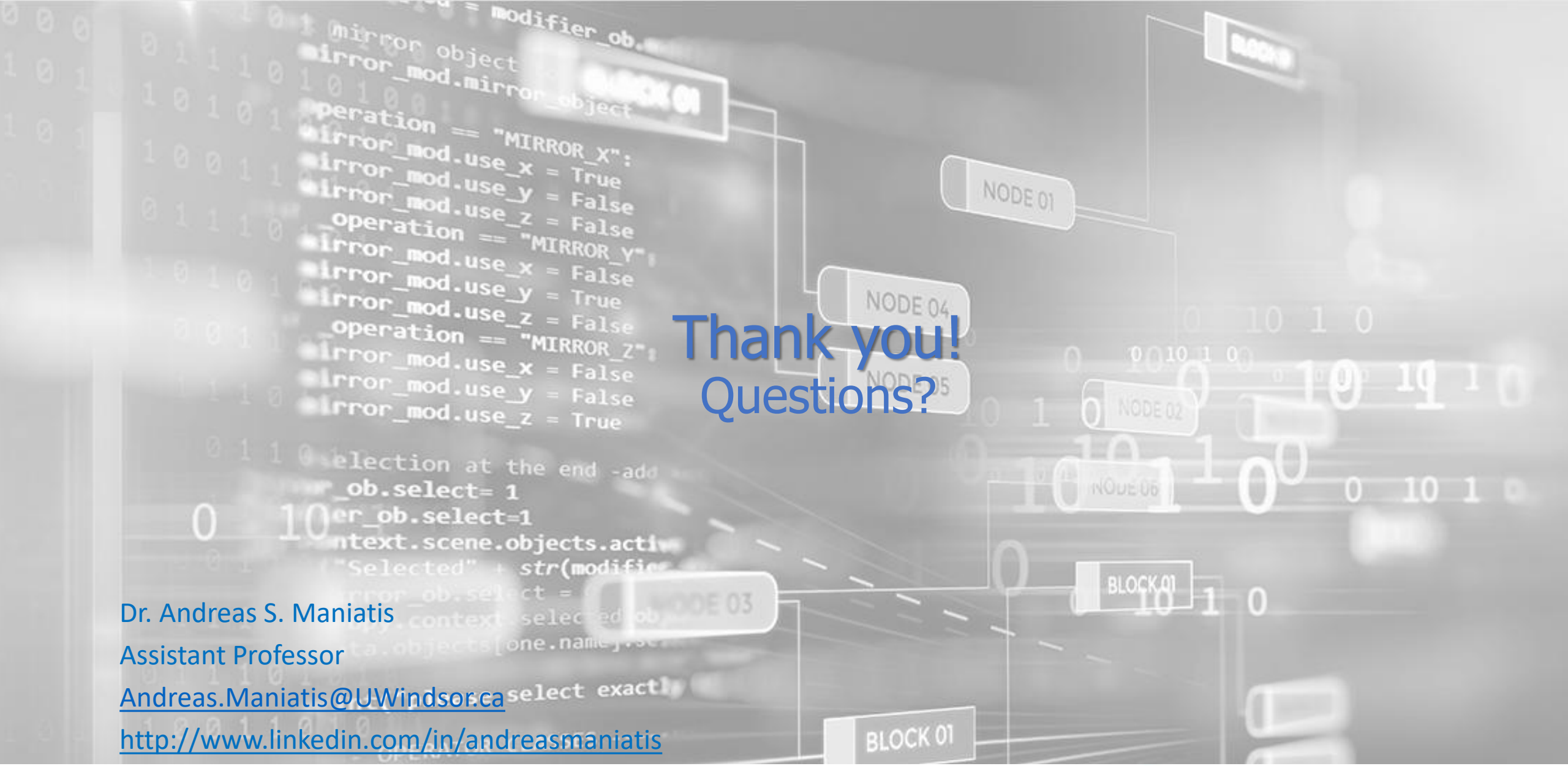
Part III
Resources

University of Windsor

Useful Resources

Useful Resources

- Download free sprite sheets: https://kenney.nl/
- Graphics Programming in Java Book: https://github.com/lwjglgamedev/vulkanbook
- Game Development in java: https://jdriven.com/blog/2020/10/GameDevelopment-in-Java



Thank you!

Questions?

Dr. Andreas S. Maniatis
Assistant Professor
Andreas.Maniatis@UWindsor.ca
http://www.linkedin.com/in/andreasmaniatis

University of Windsor